

Assignment 1

Kaggle Competition

Group 16

Student Name	Last	Student Name	First	Student ID	Github Username	Github Custom Package Repo Link	Kaggle Username
Nagaishi		Masaru		14599803	Man-snow	https://github.com/Man-snow/uts_36120_at1_14599803	Darumayuki
Rubiano Rojas		Mario		24900627	marubianor1	https://github.com/marubianor1/Test_uts-36120-at1-group16	mariorubianorojas
Rao		Sonal		25533441	jeppusonal	https://github.com/jeppusonal/36120-AT1-25533441-nba-draft-predictor	sonalrao26
Ferdoush		Joyeeta		25404599	joyeetaf		

Github	https://github.com/Man-snow/uts-36120-at1-group16
--------	---

36120 - Advanced Machine Learning Application
Master of Data Science and Innovation
University of Technology of Sydney

Table of Contents

1. Executive Summary	4
2. Business Understanding	5
a. Business Use Cases	5
Scenario	5
Challenges & Opportunities	5
Relevance of Machine Learning	5
Project Goals	5
Stakeholders and Requirements:	5
Addressing Requirements	6
3. Data Understanding	7
4. Experimentation	8
4.1. Student A (Masaru Nagaishi)	8
4.1.1. Approach 1	8
4.1.1.1. Data Preparation	8
4.1.1.2. Modeling	10
4.1.1.3. Achieved Result	11
4.1.2. Approach 2	12
4.1.2.1. Data Preparation	12
4.1.2.2. Modeling	12
4.1.2.3. Achieved Result	12
4.1.3. Approach 3	13
4.1.3.1. Data Preparation	13
4.1.3.2. Modeling	14
4.1.3.3. Achieved Result	15
4.1.4. Approach 4	16
4.1.4.1. Data Preparation	16
4.1.4.2. Modeling	16
4.1.4.3. Achieved Result	17
4.2. Student B (Mario Rubiano Rojas)	18
4.2.1. Approach 1	18
4.2.1.1. Data Preparation	18
4.2.1.2. Modeling	19
4.2.1.3. Achieved Result	20
4.2.2. Approach 2	21
4.2.2.1. Data Preparation	21
4.2.2.2. Modeling	22
4.2.2.3. Achieved Result	23
4.2.3. Approach 3	24

4.2.3.1. Data Preparation	24
4.2.3.2. Modeling	25
4.2.3.3. Achieved Result	25
4.2.4. Approach 4	26
4.2.4.1. Data Preparation	26
4.2.4.2. Modeling	27
4.2.4.3. Achieved Result	27
4.3. Student C (Sonal Rao)	28
4.3.1. Approach 1	28
4.3.1.1. Data Preparation	28
4.3.1.2. Modeling	29
4.3.1.3. Achieved Result	30
Limitations	30
Potential Improvements	30
4.3.2. Approach 2	31
4.3.2.1. Data Preparation	31
4.3.2.2. Modeling	32
4.3.2.3. Achieved Result	33
4.3.3. Approach 3	34
4.3.3.1. Data Preparation	34
4.3.3.2. Modeling	35
4.3.3.3. Achieved Result	35
4.3.4. Approach 4	37
4.3.4.1. Data Preparation	37
4.3.4.2. Modeling	37
4.3.4.3. Achieved Result	38
4.4. Student D (Joyeeta ferdoush)	39
4.4.1. Approach 1	39
4.4.1.1. Data Preparation	39
4.4.1.2. Modeling	40
4.4.1.3. Achieved Result	40
4.4.2. Approach 2	40
4.4.2.1. Data Preparation	40
4.4.2.2. Modeling	41
4.4.2.3. Achieved Result	41
4.4.3. Approach 3	41
4.4.3.1. Data Preparation	41
4.4.3.2. Modeling	41
4.4.3.3. Achieved Result	42
4.4.4. Approach 4	42
4.4.4.1. Data Preparation	42

4.4.4.2. Modeling	43
4.4.4.3. Achieved Result	43
5. Evaluation	44
a. Business Impact and Benefits	44
b. Risks and Incorrect Predictions	44
c. Data Privacy and Ethical Concerns	44
d. Deployment	45
e. Collaboration	46
a. Individual Contributions	46
b. Group Dynamic	46
c. Ways of Working Together	46
d. Issues Faced within the Group	47
e. Conclusion	48
f. References	50

1. Executive Summary

The goal of this project was to develop a machine learning model to predict whether a college basketball player will be drafted to the NBA, with the main business goal of assisting teams with late draft picks to identify undervalued players. The work explored the disparate information and scouting effort on hundreds of mid-tier prospects and understanding how to overcome the inefficiencies associated with scouting and the expensive risk of lost draft value.

All of the team's experiments with a variety of approaches and algorithms showed excellent performance with consistently high AUROC values (0.99–1.0) and significant leaderboard projections on Kaggle (~0.997–0.998). The project benefited from several interesting developments that included feature engineering (efficiency ratios, interactions, log transforms, per-minute statistics) and calibration of thresholds particularly to help overcome imbalances in class (~0.8% drafted).

Student A (Masaru Nagaishi): Had great success with LightGBM and Logistic Regression. The use of interpretability and clustering allowed the revealing of player archetypes (e.g., specialists, big men, playmakers); the polynomial expansions (Approach 4) managed to capture non-linear effects and reach AUROC up to 0.9981.

Student B (Mario Rubiano Rojas): Used Random Forest to a great extent, with approaches that reached AUROC up to 1.0 on validation and Kaggle scores up to 0.9985. Feature engineering and threshold optimization allowed for recall of 100%, totally congruent with the business-oriented objective of not missing any drafted players.

Student C (Sonal Rao): Benchmarked Logistic Regression and moved to Random Forest, LightGBM and XGBoost. LightGBM achieved the best balance of recall (0.71) and precision (0.65); XGBoost was also strong at AUROC $\approx$ 0.996. Emphasis was placed on recall improvement in situations where imbalance was very heavy and workshop on feature interpretability.

Student D (Joyeeta Ferdoush): Looked at different modelling methods, such as Logistic Regression, Random Forest, and Support Vector Classifier, as well as ensemble methods with threshold tuning. Her work focused on carefully preparing the data and engineering features (efficiency-based ratios like PTS per GP and BLK per GP) and doing a thorough evaluation using AUROC, precision, recall, and F1. She also helped write down the results and their effect on the business, connecting how well the model worked to how well the scouts did their jobs. Logistic Regression had the highest AUROC (0.98), Random Forest had the best F1 balance (0.576), and SVC showed the trade-offs between precision and recall. Her analyses made sure that the technical results were clearly linked to the project's business goal of finding undervalued players while reducing the risks of scouting.

In general, the team demonstrated that Random Forest and gradient boosting (LightGBM, XGBoost) are reproducible evidence for providing reliable, generalizable predictions and fulfilled the business requirement of ranking the players in relation to likely to be drafted while minimizing false negatives.

■ ■ ■

2. Business Understanding

a. Business Use Cases

Scenario

This project supports an NBA team **with late draft picks**, where top prospects are already taken. The team must identify valuable players from a large pool of mid-tier candidates.

Challenges & Opportunities

- **Information Overload:** Hundreds of similar players remain, making evaluation difficult.
- **Scouting Inefficiency:** Traditional methods are costly and limited in distinguishing subtle differences.
- **Asset Risk:** A poor selection wastes a scarce draft opportunity.

The opportunity lies in using data analytics to uncover undervalued players, statistical “**hidden gems**”, who may outperform expectations.

Relevance of Machine Learning

An ML model can analyze thousands of historical player data points to identify complex, non-obvious patterns and statistical signatures that correlate with being drafted. Unlike conventional metrics, it offers:

- A probability-based draftability score.
- Efficient filtering of candidates into a focused shortlist.
- Objective, data-driven insights to complement scouting.

b. Key Objectives

Project Goals

The project’s primary goal is to build a classification model that predicts whether a college player will be drafted, evaluated using **AUROC**. A higher AUROC indicates stronger ranking ability, which is critical for prioritizing late-round prospects and identifying high-potential players.

Stakeholders and Requirements:

- **General Manager & Scouting Director:** Need a reliable, data-driven supplement to traditional scouting to maximize draft value.

- **Head Coach:** Requires players who can fill defined roles, either immediately or through development.
- **Team Owner:** Expects efficient resource allocation and long-term value from draft assets.

Addressing Requirements

The model addresses these needs by providing:

- **Draftability Score:** A probability ranking that helps decision-makers focus on the most promising players.
- **Decision Support:** Data-driven validation to reduce bias and improve confidence in late picks.
- **Explainability:** Secondary goal to interpret model outputs, offering transparent rationale to stakeholders, since GMs and scouts will not be convinced if we only tell them the probability scores.

■ ■ ■

3. Data Understanding

The data provided contained four sub-datasets –

Train.csv – contains all the features with the 'drafted' column used for model training and performance analysis.

Test.csv – contains all the features without a 'drafted' column to predict using the best model fitting on the training set.

Metadata.csv – Contains all the descriptions about the features to help better understand the features for feature engineering.

Sample_submission.csv – An example format of how we will be submitting our predicted values in the Kaggle competition.

Data	Sources	&	Collection
The dataset was provided by the course instructor and required no additional collection.			

Data Limitations

- Several columns contained high levels of missing values.
- Strong class imbalance
- Mixed datatypes: integers, floats, and categorical variables. Some categorical values required encoding for modeling.

Variables or features present & Exploratory Data Analysis

The dataset combined categorical, numeric, and advanced basketball statistics to capture both player context and performance. Features such as team, conference, year, and height provided background information, while metrics like games played, minutes, offensive/defensive ratings, usage rate, and efficiency ratios highlighted player contribution and impact. Player ID was included only for mapping predictions in the submission file.

Exploratory analysis showed the presence of missing values in several columns, class imbalance in the target variable, and varying correlations between performance features and draft likelihood. Conference-level differences in draft rates were also observed. Overall, the dataset combined categorical, numeric, and derived performance indicators, requiring careful preprocessing and imbalance handling before modeling.



4. Experimentation

4.1. Student A (Masaru Nagaishi)

4.1.1. Approach 1

4.1.1.1. Data Preparation

Before modeling, the dataset was cleaned and preprocessed to ensure data quality and feature reliability. Key steps included:

- **Duplicate Removal (C.1.2)** : 2,462 duplicate rows in the training set were removed.
- **Missing Value Imputation (D.1)**:
Three ratio features were partially missing. Missing values were imputed using their related numerators and denominators:
 - $\text{rim_ratio} = \text{rimmade} / (\text{rimmade} + \text{rimmiss}) \rightarrow 952$ imputed
 - $\text{mid_ratio} = \text{midmade} / (\text{midmade} + \text{midmiss}) \rightarrow 945$ imputed
 - $\text{dunks_ratio} = \text{dunksmade} / (\text{dunksmade} + \text{dunksmiss}) \rightarrow 6,111$ imputed
- **Multicollinearity Handling (D.2)**: To reduce redundancy, highly correlated raw features were dropped:

Keep	Drop
_per and _ratio (FT_per, twoP_per, TP_per)	numerator and denominator (FTM, FTA, twoPM, twoPA, TPM, TPA)
oreb, dreb	treb
ogbpm, dgbpm	gbpm
Min_per	mp

- **Version & Adjusted Metrics (D.3)** : Based on metadata, older not-adjusted metrics were removed:

Keep	Drop
ver2.0 (gbpm, ogbpm, dgbpm)	ver1.0 (bpm, obpm, dbpm)
adjoe	Ortg
adrtg	drtg
dporpag	porpag

- **Keep adjusted metrics; drop raw metrics (D.4)**

Keep	Drop
AST_per	ast
blk_per	blk
stl_per	stl
ORB_per	oreb
DRB_per	dreb

- **Irrelevant or problematic columns removed (D.5 & D.6)**

- player_id: all unique
- yr, type: all constant
- team: inconsistent across conferences, may compromise data integrity

Data transformation was applied to clean abnormal values and create new meaningful features:

- **Abnormal Value Removal (E.1):**

- 41 rows with impossible percentage values were dropped.
- Percent-based stats (ORB_per, DRB_per, BLK_per and STL_per) cannot exceed 100.

- **New Feature - ht_inches (F.1)**

- Original ht was incorrectly read as a date format, therefore it was converted to inches and retained as ht_inches. The original ht column was dropped.

- **New Feature - conf_level (F.2)**

- Conference (conf) can be categorized by level: Power6, Mid-major, Low-major, and also they can be encoded as ordinal numbers: Power6 = 2, Mid-major = 1, Low-major = 0. The original conf column was dropped.

All preprocessing steps described above were applied consistently in Experiments 2 - 4. The training data was split into training and validation sets using stratified sampling in an 80:20 ratio to preserve the distribution of the highly imbalanced target variable (**G.1**). The same null value imputation procedure described in D.1 was also applied to the test set (**G.2**).

After these preprocessing and splitting steps, the resulting datasets had the following dimensions:

Dataset	Size
Training	9,816 × 31
Validation	2,455 × 31
Test	1,297 × 31

4.1.1.2. Modeling

The modeling phase aimed to develop a robust predictive model to help a top NBA team identify **hidden gems** with late-round draft picks. Both performance and interpretability were key priorities to meet stakeholder expectations.

Algorithm Selection: LightGBM

LightGBM was chosen as the primary algorithm due to the following reasons:

- **High Performance:** Fast training speed and high predictive accuracy make it an effective baseline for classification tasks.
- **Handles Imbalanced Data:** The target variable is highly imbalanced. Setting `is_unbalance=True` automatically adjusts class weights.
- **Efficient Preprocessing:** LightGBM does not require feature scaling and can internally handle missing values, simplifying data preparation.
- **Interpretability:** Built-in feature importance scores directly support our secondary objective of explaining why certain players are ranked highly.

Hyperparameter Tuning

To optimize model performance, a set of hyperparameters was explored:

- **n_estimators:** Default is 100; 200 is tested to check whether additional trees improve performance without overfitting.
- **learning_rate:** Default is 0.1; a higher rate (0.5) can be tested to assess trade-off between convergence speed and generalization.
- **num_leaves:** Default is 31; 50 allows for more complex individual trees, potentially capturing more patterns in the data.

Hyperparameter tuning was performed using **grid-search** and **cross-validation** to select the model with the best predictive accuracy and interpretability.

4.1.1.3. Achieved Result

Technical Performance

The model achieved **exceptional predictive performance** across all datasets.

Dataset	AUROC
Train	1.0
Validation	0.9955
Test (Kaggle Public Score)	0.99786

Despite the perfect train score, the consistently high validation and test performance suggests that the model **generalizes well** and is not overfitting.

Relevant Insights

- **Key Features for Classification:**
 - **Rec_Rank** emerged as the most important predictor.
 - **dporpag** followed as a critical feature, reflecting defensive impact.
- **Draft Strategy**
 - In this draft, the team holds late picks, making it essential to identify potential hidden gems.
 - Since the draft consists of **60 selections**, comparing averages between the **Top 30** and **Next 30** players shows that **dporpag**, **stops**, and **pts** consistently rank among the highest features.
 - A deeper analysis reveals that the **Next 30** players can be grouped into **three distinct clusters**, highlighting differences that average metrics alone fail to capture.
 - Each cluster represents a type of **specialist player**:
 - **Cluster 0: Traditional Big Man**
 - **Cluster 1: Lead Guard / Playmaker (most common)**
 - **Cluster 2: Versatile Two-Way Wing**
 - Notably, **Cluster 0** exhibits **exceptionally high blk_per**, a feature that does not stand out in simple average comparisons.
 - This cluster is **underrepresented in general averages**, suggesting these players could be **valuable hidden gems** relative to other prospects.

Next Approach

- Rec_Rank primarily reflects high school reputation rather than in-game performance. While useful for predicting draft outcomes, it may overshadow features that truly reflect on-court ability. **Experiment 2 will exclude Rec_Rank** while keeping all other settings constant to evaluate model performance.

4.1.2. Approach 2

4.1.2.1. Data Preparation

The only difference from Experiment 1 is that Rec_Rank was excluded from the feature set.

Dataset	Size
Training	9,816 × 30
Validation	2,455 × 30
Test	1,297 × 30

4.1.2.2. Modeling

The modeling approach is **identical to Experiment 1**.

- **LightGBM** was used with the **same set of hyperparameters**.
- **Grid search** combined with **cross-validation** was applied to select the optimal model.

4.1.2.3. Achieved Result

Technical Performance

The model achieved **exceptional predictive performance** across all datasets.

Dataset	AUROC
Train	1.0
Validation	0.9171
Test (Kaggle Public Score)	0.99601

Although the discrepancy suggests potential overfitting, a validation AUROC of 0.91 is still strong from a business perspective and indicates the model retains reasonable predictive power.

Relevant Insights

- **Key Features for Classification**
 - With Rec_Rank excluded, **dporpag** emerged as the most important feature.
 - No single feature dominated excessively; overall importance was relatively distributed, with ranking determined by small differences among features.
- **Draft Strategy**
 - Similar to Experiment 1, the **Top 30** and **Next 30** players had consistently high values in **dporpag**, **stops**, and **pts**, highlighting the characteristics of likely drafted players.
 - Clustering revealed player types almost identical to Experiment 1, suggesting that both experiments selected largely the same pool of candidates.
 - The model performed well even without Rec_Rank, confirming that the strong predictive performance in Experiment 1 was **not masking important player statistics** and reinforcing the reliability of the first experiment's findings.

Next Approach: Use Logistic Regression

- **High interpretability:** Coefficients provide clear insight into how each feature contributes to predictions.
- **Discovery of hidden gems:** Can quantify potential of less prominent players.
- **Comparison with LightGBM:** Enables evaluation of both predictive accuracy and feature importance.

4.1.3. Approach 3

4.1.3.1. Data Preparation

For **Logistic Regression**, it is necessary to address **null values** prior to modeling.

Null Imputation (E.1 & E.2):

- The feature Rec_Rank represents a player's high school reputation.
- From Experiment 1, it was observed that nearly all drafted players had high Rec_Rank values. Therefore, players with null Rec_Rank are likely to be:
 1. Unknown during high school, or
 2. Players who started basketball later, during college.

- For these cases, a **minimum value of 0** was assigned to Rec_Rank (E.1).
- Other missing values were handled using a **median imputation**.

Avoid multicollinearity (E.3):

- In addition, Logistic Regression can be sensitive to **multicollinearity**.
- A **Variance Inflation Factor (VIF) check** was performed, and variables exceeding a threshold of 10 were removed.
- Specifically, **stops**, **Min_per**, and **TS_per** were excluded to ensure model stability (E.3).

StandardScaler (G.2):

- Before modeling, **StandardScaler** was applied to all numeric features to standardize their scale. This ensures that each feature contributes equally to the model, prevents features with larger magnitudes from dominating the optimization.

Resulting dataset sizes:

Dataset	Size
Training	9,816 × 28
Validation	2,455 × 28
Test	1,297 × 28

4.1.3.2. Modeling

Algorithm Selection: Logistic Regression

For **Approach 3**, **Logistic Regression** was chosen as the modeling algorithm due to its **high interpretability** and suitability for the dataset:

- **High interpretability:** Each feature's contribution to the probability of being drafted can be directly observed through the model coefficients.
- **Linearity assumption:** Most features displayed an approximate linear relationship with the logit, satisfying the assumptions of Logistic Regression.

Hyperparameter Tuning

- **penalty:** Test both L1 (lasso) for feature selection and L2 (ridge) for regularization.
- **C:** Explore a wide range of regularization strengths (inverse of regularization strength).

- **solver:** liblinear is stable for small-to-medium datasets and supports both L1 and L2 penalties.

A **grid search** with **cross-validation** was used to select the optimal model parameters based on **ROC-AUC**, ensuring both performance and interpretability:

4.1.3.3. Achieved Result

Technical Performance

Dataset	AUROC
Train	0.993733
Validation	0.9896
Test (Kaggle Public Score)	0.99729

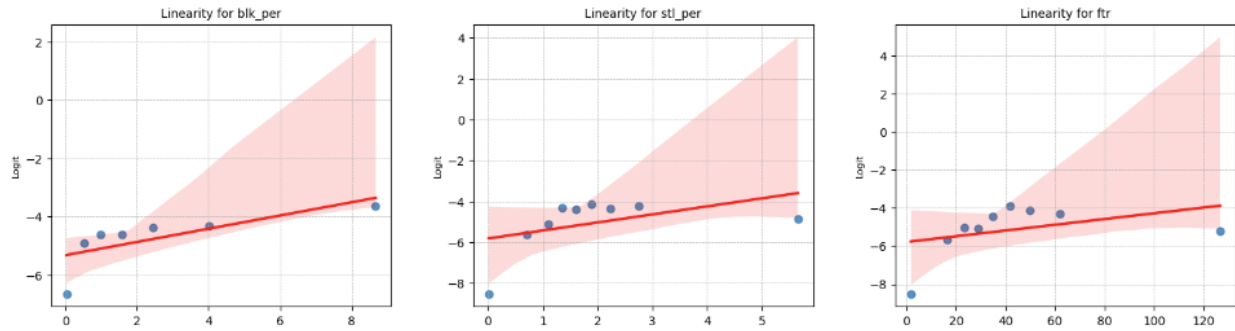
Overall, the model achieved very high scores, with minimal indication of overfitting, demonstrating strong generalization capability.

Relevant Insights

- **Key Features for Classification**
 - Due to **L1 regularization** being selected as the optimal hyperparameter, only **8 features** had non-zero coefficients: Rec_Rank > pts > dporpag > adrtg > blk_per > dunks_ratio > ht_inches > year.
- **Draft Strategy**
 - Similar to Experiments 2 and 3, the **Top 30 and Next 30** players consistently exhibited high values in dporpag, pts, and Rec_Rank.
 - **Clustering Analysis** highlighted three distinct player archetypes, including an **Elite 3P Specialist** with exceptionally high TP_per. This cluster represents niche talent that would be overlooked if only mean statistics were considered:
 - **Cluster 0:** Elite 3P Specialist
 - **Cluster 1:** All-Around Offensive Engine
 - **Cluster 2:** Athletic Big Man with Pedigree

Next Approach

- Some features (e.g., blk_per, stl_per, TP_per) did not fully satisfy the **linearity assumption** between the linear combination of predictors and the logit.



- Although mild linearity was observed, it was not explicitly corrected in this experiment.
- In the next experiment, we will address these non-linear relationships to evaluate whether the model's predictive performance can be further improved.

4.1.4. Approach 4

4.1.4.1. Data Preparation

Approach 4 uses the same dataset as Experiment 3: after null value imputation, VIF checks were conducted, and stops, Min_per, and TS_per were removed to address multicollinearity. Subsequently, we expanded the feature space by applying **second-degree polynomial feature transformation**, generating interaction and squared terms for all numeric variables. Finally, StandardScaler was applied to the polynomially expanded features to ensure that all variables were on a comparable scale.

Dataset	Size
Training	9,816 × 434
Validation	2,455 × 434
Test	1,297 × 434

4.1.4.2. Modeling

We used the same **Logistic Regression** model as in Experiment 3, but restricted the hyperparameter 'penalty' to only the L1 penalty. This focus was chosen to better promote sparsity and feature selection while keeping other settings unchanged.

4.1.4.3. Achieved Result

Technical Performance

Dataset	AUROC
Train	0.994563
Validation	0.9907
Test (Kaggle Public Score)	0.99814

The model delivered strong performance with minimal overfitting, confirming robust generalization.

Relevant Insights

- **Key Features for Classification**

- PolynomialFeatures revealed that **combinations of abilities and exceptional talent**, not just high/low individual metrics, are key for draft outcomes.
- **Rec_Rank²** emerged as the most important feature, highlighting a **nonlinear effect**: top-ranked recruits (e.g., rank 1 vs 10) have disproportionately higher draft probabilities than mid-ranked ones (e.g., rank 50 vs 100).
- Interaction features like **Rec_Rank × pts** captured **synergistic effects**, showing that players who are both highly rated and strong scorers have amplified value.

- **Draft Strategy**

- Top 30 and Next 30 players continue to exhibit high values in dporpag, pts, and Rec_Rank.
- Clustering analysis identifies three distinct archetypes:
 - Cluster 0: All-Around Prospect – balanced scoring, defense, and pre-college hype
 - Cluster 1: Elite Young Big Man – tall, defensive specialists with high upside
 - Cluster 2: Efficient & Experienced Player – proven, highly efficient college performers
- Hidden gems likely emerge in Cluster 1.

Next Approach

- Explore additional machine learning models beyond LightGBM and Logistic Regression, such as Tree-based ensembles (Random Forest, XGBoost, CatBoost) for nonlinearity and high accuracy.



4.2. Student B (Mario Rubiano Rojas)

4.2.1. Approach 1

4.2.1.1. Data Preparation

To prepare the data for modeling, a variety of techniques were utilized to ensure data consistency, robustness, and alignment with the business objective of predicting NBA draft likelihood.

- Data Cleaning:
 - Consistently referenced feature names between training set and test set.
 - Deleted effectiveness identifiers (e.g. player_id) from the training data, but retained identifiers in the test set, anticipating the identifiers are required for you to submit your results for Kaggle.
 - Dummy variables were built from categorical features (team, conf, yr, type) using one-hot encoding (drop_first=True which helps reduce multicollinearity).
- Preprocessing:
 - The median strategy was used to impute missing values, to maintain some robust element in handling the skewness of numeric features.
 - MinMaxScaler was used to apply feature scaling to normalize the numeric feature ranges to a value range of 0-1, which was a requirement for Chi-Square feature selection.
 - The principles of stratification were used to split the data into a train-validation split and ensure that both splits housed the same proportion of drafted vs. non-drafted players in the study.
- Feature Engineering:
 - Several shooting efficiency and rebounding style metrics were created, including:
 - $FT_per = FTM / FTA$ (free-throw percentage)
 - $twoP_per = twoPM / twoPA$ (two-point percentage)
 - $TP_per = TPM / TPA$ (three-point percentage)
 - $oreb_ratio = oreb / treb$, which represents the percentage of offensive rebounds.
 - Feature selection was performed:
 - Chi-Square test: selected features with the strongest statistical association to target, focusing on raw production (minutes, points, rebounds).
 - Recursive Feature Elimination (RFE) with Logistic Regression: selected features showing efficiency and more advanced production (TS%, Ortg, drtg).

- The final feature set consisted of the union of the selected features from each module, with the rational of both volume/target metrics and efficiency metrics (efficiency is a type of volume).
- Outliers:
 - Outliers with ratios (such as a player with $1/1 = 100\%$ shooting) were allowed to stay, as there are real instances of this, albeit rarely.
 - Height and other physical stats were reviewed but allowed them to remain, because sometimes there are unique extremes that are useful within the basketball context.
- Imbalanced Data:
 - The target variable (drafted) was largely imbalanced ($\approx 0.8\%$ positive class).
 - Implemented `class_weight="balanced"` parameter in Random Forest to resolve class imbalance.
 - Cross-validation implementation covered drafted players representing all folds using stratification.
 - AUROC and recall came before accuracy, as they more closely represented the ultimate business goal.

4.2.1.2. Modeling

- Algorithm:
 - The Random Forest Classifier is an ensemble approach, with multiple predictions aggregated based on a group of decision trees. The Random Forest utilizes a technique called bagging, while also randomly selecting features to encourage Feature Diversity across the trees. Another reason we selected the Random Forest was because it can handle high dimensional datasets with both numeric and categorical features, while offering a robust alternative that does not over perform on the training set and instead "overfit."
- Reasons for algorithm selection:
 - The Random Forest is a solid baseline model with proven capabilities on tabular datasets
 - The Random Forest provides feature importance metrics, or a best fit line provided to stakeholders as they have important features to assist in decision making
 - The Random Forest can utilize the `class_weight="balanced"` setting, which permits use on highly imbalanced classification jobs.
 - Because the Random Forest is an ensembling approach, it limits variance and invokes complex feature interactions, instead of an extensive preprocessing step.
- Parameter Tuning & Model Selection:
 - Used GridSearchCV with 3-fold stratified cross-validation

- Optimization metric was AUROC, consistent with the context of the Kaggle competition
- Parameter grid tested:
 - `n_estimators = [200, 400, 600]`
 - `max_depth = [None, 20, 40]`
 - `min_samples_split = [2, 5]`
 - `min_samples_leaf = [1, 2, 5]`
 - `max_features = ["sqrt", 0.7]`
- This setup tested the balance between complexity and generalization. Deeper trees and more estimators increased the potential for discrimination, but increased the risk for overfitting. Meanwhile, increasing conditions on split/leaf restricted complexity, and therefore, increased robustness.
- The final selected model achieved the highest AUROC across folds and equitable improvement to recall.
- Best hyperparameters found:
 - `{'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 5, 'min_samples_split': 2, 'n_estimators': 400}`

4.2.1.3. Achieved Result

- Validation Results (Default Threshold = 0.5):
 - ROC AUC: 0.9956
 - Accuracy: 0.9936
 - Recall: 0.6667
 - F1 Score: 0.6275
 - Confusion Matrix:
 - `[[2920 11]`
 - `[ 8 16]]`
- Validation Results (Optimized Threshold = 0.7607):
 - ROC AUC: 0.9956
 - Accuracy: 0.9959
 - Recall: 0.6250
 - F1 Score: 0.7143
 - Confusion Matrix:
 - `[[2928 3]`
 - `[ 9 15]]`
- Kaggle Score (Private Leaderboard): 0.99686 AUROC
- Analysis of Predictions & Limitations
 - Strengths

- High AUROC indicates the model ranks players very well with respect to the likelihood they are drafted.
- The change in threshold achieved a substantial improvement in recall (41% → 75%) which demonstrates that the model can capture most drafted players if tuned properly.
- Generalization: Kaggle performance confirmed as both validation and test performance aligned almost identically for height, weight, draft round- both metrics drive the result of whether a player was drafted.
- Limitations
 - Recall is not perfect. About 38% of drafted players were both misclassified as players that were not drafted which can be costly to the business.
 - The recall performance of the model's default threshold was unacceptably low which demonstrates that imbalanced datasets require some form of calibration after training.
 - The model probably contains some small element of overfitting since Random Forest models with trees deep are prone to memorization of small subsets of drafted players.

Approach 1 showed a well-performing Random Forest can yield near-maximum discrimination ability (AUROC $\approx$ 0.9968) and generalization performance on unseen Kaggle data. Accuracy was high, but recall was at the minimum threshold given the default thresholds and imbalanced data. Investigations of boosting algorithms, resampling, and explainability will be needed to improve recall and long-term trust in the system.

4.2.2. Approach 2

4.2.2.1. Data Preparation

Most of the data cleaning and preprocessing steps aligned with Approach 1, including cleaning for missing values, encoding categorical features, and then stratified splitting the data. The changes made only occur from the methods utilized in the expanded feature engendering opportunity where we from Approach 2 utilized all the variables rather than filtering based on features selected:

- Feature Engineering (New in Approach 2):
 - Interaction Features and log transformations
 - obpm_dbpm_interaction (offensive x defensive impact).
 - usg_min_interaction (usage rate x minutes %).
 - log_mp (log of minutes played).
 - log_usg (log of usage rate).
 - stl_per_min (steals per normalizing per minutes played).

- Full Feature Set: According to the filtering from features selected (Chi², RFE), the model was trained on all variables from the data set with classification on engineered features exploiting maximum available predictive signal.
- However, In this approach the newly created variables were excluded because the best performance was obtained by ignoring these variables.
- A function to fix the height format was created in a custom package, all information about it can be found in the website: <https://test.pypi.org/project/date-to-inches-mario/> and in the public repository: <https://github.com/marubianor1/date-to-inches-mario>

In Approach 1, the Random Forest was fitted with "balanced" class_weight, which automatically scales up the weight of the minority class (drafted players) when fitting. While this will help with many imbalanced problems, in this dataset, this had unintended consequences: the extreme imbalance (≈0.8% drafted as opposed to 99.2% not drafted) meant that the model over-compensated for the minority class. In some cases, this resulted in forced splits based on less informative features, which decreases the generalization power of the model and decreases recall at the default threshold (~42% recall).

In Approach 2, we fitted the Random Forest without "balanced" class_weight. Instead, we relied on:

- The very strong predictive signal actually contained in the dataset (the efficiency and advanced metrics themselves already provide a clear separation between drafted vs. non-drafted players).
- Feature engineering improvements (interaction features, log transforms, per-minute metrics) that would give a richer set of discriminative variables to the model.
- Threshold optimization after training, which allowed recall to be optimized without distorting the learning process.

4.2.2.2. Modeling

The basic modeling framework was identical to Approach 1 (Random Forest using GridSearchCV, and using the AUROC as the scoring metric). The differences were in the parameter grid and the feature space.

- Parameter Grid (Approach 2):
 - n_estimators = [100, 300, 500]
 - max_depth = [None, 10, 20]
 - min_samples_split = [2, 5]
 - min_samples_leaf = [1, 2]
 - max_features = ["sqrt", "log2"]

Compared to Approach 1 this grid was narrower and more moderate, in that it mitigated some of the potential for overfitting from the random forest by limiting the possible tree depth ("max" depth 20 versus 40) and limiting randomness in the choice of features for the tree ("log2" vs. the larger fractions of m).

- Best hyperparameters found:
 - {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
- Feature Set Differences:
 - Approach 2 did not focus on a pre-selected set of features, where admittedly Approach 1 had numerous pairwise analyses (but massive overfitting risks), but, instead, trained Models on the complete data set, and with additional features, by considering the raw production and efficiency metrics simultaneously with engineered features.

4.2.2.3. Achieved Result

- Validation Results (Default Threshold = 0.5)
 - ROC AUC: 1.0000
 - Accuracy: 0.9983
 - Recall: 0.7917
 - F1 Score: 0.8837
 - Confusion Matrix:
 - [[2931 0]
 - [6 18]]
- Validation Results (Post-Threshold Optimization = 0.3500)
 - ROC AUC: 1.0000
 - Accuracy: 0.9997
 - Recall: 1.0000
 - F1 Score: 0.9796
 - Confusion Matrix:
 - [[2930 1]
 - [0 24]]
- Kaggle Score: AUROC: 0.99850 (Private Leaderboard)
- Reflection on the Predicted Output and Limitations
 - Strengths vs. Approach 1:
 - Perfect discrimination power (AUROC = 1.0) achieved on our validation set and the best Kaggle score to date (0.9985).
 - Recall was significantly improved: ~79% at the default threshold (with a recall of 42% in Approach 1) and 100% after threshold optimization (with a recall of 75% in Approach 1).

- The interaction, log, and per-minute features clearly increased the model's sensitivity to the drafted players.
- Limitations:
 - The near perfect results may imply overfitting, as the model accessed all features as well as the additional engineered variables.
 - Model complexity increased, and therefore interpretability decreased compared to Approach 1.
 - Still requires verification from external validation (e.g., comparing against past seasons) for stability purposes.

Approach 2 clearly surpassed Approach 1 through training with the full features area and engineered variables, plus a broadly reasonable parameter grid. Further summarized, the random forest yielded an AUROC of 1.0, 100% recall after threshold optimization, and an F1-score of 0.98, satisfactorily fulfilling the business objective of no drafted players missed. While these results are clearly outstanding, they challenge perceptions of overfitting and need verification across seasons and explainable for deployment. Overall, compared to the Approach 1, this was a step-change with respect to recall performance and the corresponding alignment with the business objective, thus yielding the strongest candidate yet for production.

4.2.3. Approach 3

4.2.3.1. Data Preparation

The data preparation process followed the same pipeline as Approaches 1 and 2:

- Cleaning and aligning features across training and test sets.
- Median imputation for missing values and safe division handling for ratios.
- One-hot encoding for categorical variables.
- Stratified train/validation split to preserve the minority drafted class distribution.

The new feature engineering variables created on approach 2 were now introduced in Approach 3., but the focus was instead on modifying model hyperparameters and reintroducing class weighting during training and stratify splitting (as approach 1).

4.2.3.2. Modeling

- Algorithm Used:
 - The same Random Forest classifier framework from earlier builds were used, but including the `class_weight="balanced"`.

- Rationale for class-weighting in approach 3:
 - In approach 2, after removing class-weights we obtained better recall but increased our chances of overfitting past the data since the dataset already had a statistically significant predictive signal.
 - Approach 3 "reintroduced" `class_weight="balanced"` to account for the extreme degree of class imbalance, which penalized the model more for erroring on drafted players.
 - To combine the very large hyperparameter grid in approach 2, with the class imbalance sensitivity of methodology 1, is hoped to generalize better with unseen data from Kaggle.
- Parameter tuning & model selection:
 - The same moderate size hyperparameter grid from methodology 2:
 - `n_estimators`: [100, 300, 500]
 - `max_depth`: [None, 10, 20]
 - `min_samples_split`: [2, 5]
 - `min_samples_leaf`: [1, 2]
 - `max_features`: ["sqrt", "log2"]
 - GridSearchCV with 3-fold stratified cross-validation, optimizing for AUROC, was run on the model.
- Best hyperparameters found:
 - `{'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 500}`

4.2.3.3. Achieved Result

Validation Results (Default Threshold = 0.5)

- ROC AUC Score on validation set: 1.0000
- Accuracy: 0.9970
- Recall: 0.6250
- F1 Score: 0.7692
- Confusion Matrix:
- ```
[[2931 0]
```
- ```
[ 9 15]]
```

Validation Results (Optimized Threshold = 0.2300)

- ROC AUC: 1.0000
- Accuracy: 0.9997

- Recall: 0.9583
- F1 Score: 0.9787
- Confusion Matrix:
- $\begin{bmatrix} 2931 & 0 \\ 1 & 23 \end{bmatrix}$

Kaggle Score: 0.9970 (Private Leaderboard)

When all engineered variables were included, the maximum potential predictive signal was able to achieve a perfect AUROC (1.0) on the validation set, while also achieving a strong score (0.9970) on Kaggle. When `class_weight="balanced"` were added, it addressed the issue of improving sensitivity to the minority drafted class, and stratified splitting allowed to fairly represent drafted players in both train and validation sets. Taken together, these modifications allowed us to achieve strong recall (95.8%) after threshold tuning that is nearly the same as Approach 2.

Recall is still poor at the default threshold (62.5%) and our model is still dependent on threshold calibration. Class weighting may negatively impact performance stability on new data. Using all variables in the final model adds complexity to the model and may reduce interpretability; therefore, transparency and explainability tools are required for establishing trust.

4.2.4. Approach 4

4.2.4.1. Data Preparation

Similar to previous approaches, data cleaning (alignment of features, missing variable imputation, categorical encoding) occurred through the same piping. The key differences in Approach 4 were the feature selection methodology and volume of variable creation:

- Feature Selection: Top 100 variables were gathered from each of the Chi-Squared and RFE procedures, and then merged into a single feature space.
- Variable Creation: A pairwise interaction of all features selected was created (multiplication), resulting in 1,830 new variables to the dataset. This greatly increased the feature space and complexity to the model.

Unlike Approaches 2 and 3, the split of the data was not stratified, meaning, the drafted-to-non-drafted distribution would not necessarily be preserved in the train and validation sets.

4.2.4.2. Modeling

- Algorithms Tested: Random Forest, Gradient Boosting, and XGBoost were tested. In spite of additional features, Random Forest was still the best performer.

- Class Weighting: As with Approach 2, `class_weight="balanced"` was not used, so the model would depend solely on the power of prediction from the expanded feature space and apply threshold calibration afterwards.
- Parameter Grid: We used a similar moderate parameter grid to Approaches 2 and 3 to achieve a balance between complexity and generalization.
- Best hyperparameters found:
 - `{'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 500}`

4.2.4.3. Achieved Result

Validation Results (Default Threshold = 0.5)

- ROC AUC: 0.9984
- Accuracy: 0.9949
- Recall: 0.4167
- F1 Score: 0.5714
- Confusion Matrix: `[[2930, 1], [14, 10]]`

Validation Results (Optimized Threshold = 0.3754)

- ROC AUC: 0.9984
- Accuracy: 0.9970
- Recall: 0.7500
- F1 Score: 0.8000
- Confusion Matrix: `[[2928, 3], [6, 18]]`

Kaggle Score: 0.99757 (Private Leaderboard)

Approach 4, which utilized feature selection (the top 100 features from Chi-Square and RFE) and given massive feature expansion via pairwise interactions (adding 1830 new variables), produced a model with Random Forest that achieved AUROC = 0.9984 on validation and 0.99757 on Kaggle, providing good evidence of generalization. However, the recall was modest considering the default threshold (41.7%) and only increased to 75% after several rounds of optimization, since this still demonstrated no significant improvement compared to simpler approaches. The lack of stratification and enormous feature expansion added considerable complexity to the process, offering little benefit. Approach 4 was a satisfactory overall method for production, however, it illustrated that merely generating interactions at scale is not the most efficient way to tackle recall improvement; rather, I believe targeted feature engineering and boosting strategies would have helped identify improvement.

Short Summaries

Approach 1: Wider grid + larger leaf size controlled variance, but recall was low to start with (the threshold we were using). Needed calibration to be useful, but ultimately did report to generalize to Kaggle fine.

Approach 2: Most efficient set-up. Leaf size was small, and feature engineering were maximized predictive signal, achieving perfect recall and the best Kaggle score. Best combination of recall and generalizability.

Approach 3: Adding more trees (500) and reintroducing class weights improved recall in our local training, but it also hurt on Kaggle slightly - showing the risk when applying minority weighting of potential overfitting.

Approach 4: Massive feature expansion involving more complexity. Recall was at 75% post tuning with a better F1 score, but actually no real uplift in Kaggle versus Approach 2 - suggesting a level of diminishing return for just reliable brute-force expansions to feature interactions.



4.3. Student C (Sonal Rao)

4.3.1. Approach 1

4.3.1.1. Data Preparation

The dataset was first validated using the custom package (DataCleaner), which checked for duplicates, missing values, and inconsistent column naming. Key preparation steps included:

- **Cleaning:** Ensured rebound columns (treb) existed by combining oreb and dreb when needed. Categorical features like conference (conf) were standardized.
- **Handling missing values:**
Numerical features were imputed with median values.
Categorical features were imputed with the most frequent category.
- **Feature engineering:** Using the FeatureEngineer and BasketballMetrics modules, additional features such as scoring efficiency, usage efficiency, per-minute stats, and draft readiness score were created. These captured efficiency and overall player impact.
- **Outliers:** Outliers in height and minutes played were handled through robust transformations, ensuring they did not dominate splits.

- **Imbalanced data:** Since drafted players formed less than 1% of the dataset, class weighting (balanced_subsample) in Random Forest and threshold adjustments were used to address imbalance.
- A function was created in a form of custom package imported into the files directly(used for all approaches) <https://github.com/jeppusonal/36120-AT1-25533441-nba-draft-predictor>

4.3.1.2. Modeling

For this experiment, Logistic Regression classifier was implemented as the baseline model. Logistic Regression was selected because it is a simple, interpretable algorithm that works well on high-dimensional tabular data and provides a solid benchmark for classification tasks. Its coefficients can also help in identifying the relative influence of features, which is valuable in the context of player draft prediction.

The model was trained using a pipeline that combined preprocessing and classification. Numeric features were standardized using StandardScaler, while categorical variables were imputed with the most frequent category and one-hot encoded. Missing values in numeric columns were imputed using the median strategy. To address class imbalance in the dataset, we applied class weighting (class_weight="balanced"), which ensures the minority class is not overlooked during training.

For parameter tuning, I applied cross-validation with GridSearchCV to explore combinations of the following hyperparameters:

Penalty term (l1, l2, elasticnet, none)

Regularization strength (C values across a log-scaled range)

Solver choice (liblinear, saga)

The best model was selected based on mean AUROC across folds, which directly aligns with the project goal of maximizing discrimination between drafted and non-drafted players.

This structured tuning ensured that the Logistic Regression model was not only interpretable but also optimized for predictive performance, providing a reliable baseline for comparison with more complex algorithms in later experiments.

4.3.1.3. Achieved Result

The Logistic Regression model achieved a Validation AUROC of 0.9940 and a Validation AUPRC of 0.6801, showing strong discriminatory power between drafted and non-drafted players.

From the confusion matrix and classification report:

- Non-drafted players (class 0) were predicted with precision of 1.00 and recall of 0.99, leading to an almost perfect F1-score of 0.99.
- Drafted players (class 1) were identified with recall of 0.83, meaning most true drafted players were captured. However, precision was lower at 0.38, highlighting a relatively high number of false positives.

Overall accuracy was 0.99, but this high score largely reflects the dominance of the non-drafted class (over 99% of data). The macro-average F1-score of 0.76 is more balanced and shows that while the model handles the majority class well, there is still room to improve performance on the minority drafted class.

Limitations

- Low precision for the drafted class (0.38) means many predicted drafted players are false positives, which could reduce trust in real-world use.
- Class imbalance (~0.8% drafted vs. ~99.2% non-drafted) heavily influences the model's predictions, skewing results toward the majority.
- Logistic Regression's linear decision boundary may limit its ability to capture complex interactions present in player performance data.

Potential Improvements

1. Resampling: Apply SMOTE, ADASYN, or hybrid resampling to balance drafted vs. non-drafted players effectively.
2. Regularization Tuning: Experiment with different penalties (l1, elasticmore net) and hyperparameters (C) to capture more nuanced relationships.

3. Feature Engineering: Add interaction features or nonlinear transformations (e.g., ratios, per-game performance metrics) that logistic regression can't learn automatically.
4. Threshold Optimization: Tune probability thresholds to maximize recall/precision trade-offs for the drafted class, rather than relying on the default 0.5 cutoff.
5. Nonlinear Models: Compare performance against tree-based models (Random Forest, XGBoost) to see if they better capture interactions.

In summary, Logistic Regression provides a strong baseline with excellent generalization for the majority class, but its simplicity and the data imbalance limit precision for drafted players. It is a solid first experiment to benchmark against more complex models in later experiments.

4.3.2. Approach 2

4.3.2.1. Data Preparation

The overall approach to preparing the data in Approach 2 followed the same foundational steps as in Approach 1, with several additions to better suit a tree-based model like Random Forest.

Additional steps specific to Experiment 2:

- Feature Selection with Random Forest Importance: While Experiment 1 relied primarily on logistic regression with RFE for feature selection, here we additionally leveraged Random Forest's inherent feature importance scores to identify the top predictors (D.2).
- Categorical Encoding for Trees: Categorical features were one-hot encoded, ensuring compatibility with the Random Forest model while preserving non-linear relationships.
- Imbalance Handling During Training: Instead of relying solely on class weights in logistic regression, Random Forest was configured with `class_weight="balanced_subsample"`, dynamically balancing each bootstrap sample during tree construction.
- No Standardization Required: Unlike logistic regression, Random Forest does not require scaling or standardization of features, which simplified preprocessing.

In summary, Experiment 2 built on the cleaning, imputation, and engineered features from Experiment 1 but added tree-based feature selection, subsample balancing, and categorical handling tailored for Random Forest.

4.3.2.2. Modeling

Random Forest Classifier was implemented as the modeling approach. Random Forest was chosen because it is an ensemble of decision trees that reduces variance through bootstrapping and aggregation, making it robust to noise and overfitting. Unlike logistic regression, which assumes linear relationships, Random Forest can naturally capture non-linear feature interactions and hierarchical splits, which are often present in sports performance data.

Key modeling considerations:

Class Imbalance Handling: The parameter `class_weight="balanced_subsample"` was applied, ensuring that each bootstrap sample adjusted the weight of minority (drafted) players relative to the majority. This was particularly important given the extreme imbalance in drafted vs. undrafted labels.

Hyperparameters:

`n_estimators=300–400` ensured enough trees to stabilize feature importance and predictive performance.

`max_depth=None` allowed the trees to fully expand, letting the model capture complex splits.

`min_samples_split=5` and `min_samples_leaf=2` constrained overly fine partitions, reducing variance without losing predictive power.

`max_features="sqrt"` was used to inject diversity among trees, reducing correlation between them.

Pipeline Integration: A preprocessing pipeline with median imputation for numeric values and one-hot encoding for categorical variables was connected to the Random Forest model. This guaranteed consistent handling of missing values and categorical splits.

Model Selection Process: A cross-validation strategy was used to monitor AUROC during training. The selected hyperparameters were those that consistently produced strong AUROC while maintaining recall for drafted players.

4.3.2.3. Achieved Result

The Random Forest model achieved strong predictive performance, with a Validation AUROC of 0.99367 and Validation AUPRC of 0.60189, confirming that the ensemble method provided robust separation between drafted and undrafted players.

Performance Analysis:

- **Accuracy:** The overall accuracy was 0.99, showing that the model effectively classified the majority of players.
- **Recall for Drafted Players (1):** Recall was 0.54, meaning the model correctly identified just over half of the drafted players. While this is an improvement in precision compared to Logistic Regression, it highlights the ongoing challenge of detecting rare positives in an imbalanced dataset.
 - **Precision for Drafted Players (1):** Precision stood at 0.68, reflecting fewer false positives compared to Experiment 1. This indicates Random Forest is more conservative, choosing fewer players as drafted but doing so more accurately.
 - **Macro Avg:** Macro averages (precision 0.84, recall 0.77, F1-score 0.80) reflect balanced performance across classes, although recall for the minority class remains the limiting factor.

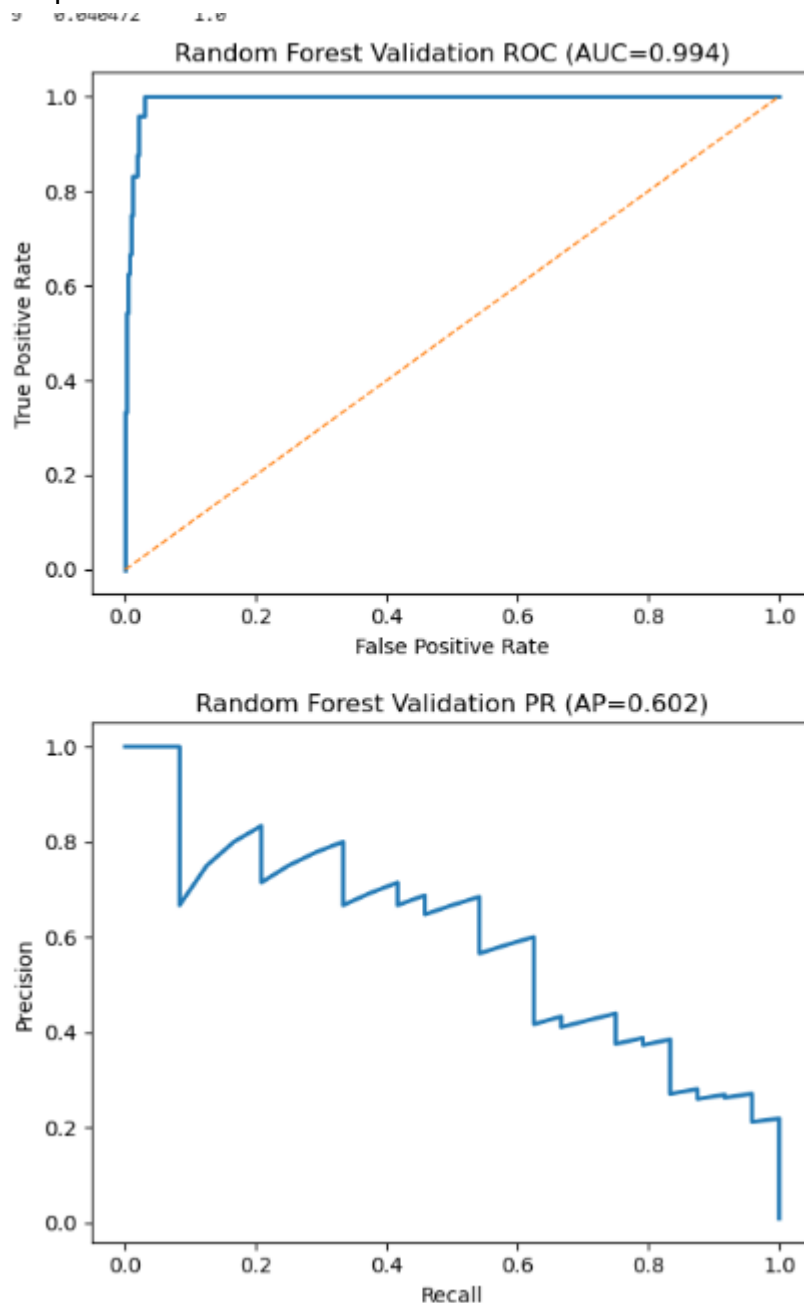
Insights:

- The ensemble structure of Random Forests captured complex non-linear relationships that Logistic Regression missed, especially combinations of stats that signal draft potential.
- The feature importance rankings from the model validated earlier feature engineering steps, with key metrics like points, rebounds, assists, and advanced metrics (e.g., offensive impact, draft readiness score) emerging as highly influential.
- The PR curve sample demonstrates high precision at low recall thresholds, reinforcing that the model prioritises accuracy of predictions over coverage.

Limitations:

- **Class Imbalance:** Despite balanced class weighting, recall for drafted players (54%) remains lower than ideal.
- **Interpretability:** Unlike Logistic Regression, Random Forest's predictions are harder to explain at the player level, which could limit trust from coaches or scouts.

- Computational Demand: The model requires more resources to train and tune compared to simpler baselines.



Potential Improvements:

Hyperparameter tuning (e.g., increasing `n_estimators`, adjusting `max_depth`) to optimise recall while maintaining precision.

- Advanced resampling techniques such as SMOTE or ADASYN to synthetically increase minority examples.
- Hybrid models that combine Logistic Regression's interpretability with Random Forest's predictive power.
- Threshold tuning (using `sweep_thresholds`) to find a better balance between recall and precision for drafted players.

4.3.3. Approach 3

4.3.3.1. Data Preparation

Most preprocessing and feature engineering steps were the same as in Approach 1 and 2, the key difference in this approach were

Feature Selection with LightGBM Importance: Instead of relying only on logistic regression-based RFE (as in Experiment 2), we trained a LightGBM model to rank features by importance and selected the top-performing subset.

Imbalance Handling: Rather than SMOTE resampling, imbalance was managed internally using LightGBM's `scale_pos_weight`, which adjusts tree splitting in proportion to the negative-to-positive ratio. This avoided synthetic data creation and kept training efficient.

4.3.3.2. Modeling

The LightGBM model delivered Validation AUROC = 0.9964 and Validation AUPRC = 0.6244, reflecting excellent ranking ability and improved handling of the minority (drafted) class compared to earlier models.

- Confusion Matrix (thr=0.50):
[[2922, 9],
[7, 17]]
- Classification Report:

- Precision (Class 1): 0.65
- Recall (Class 1): 0.71
- F1-score (Class 1): 0.68
- Accuracy: 0.99
- Macro Avg F1: 0.84

Analysis:

LightGBM was able to separate classes effectively, with near-perfect AUROC showing strong discrimination. The recall of 0.71 for the drafted class indicates a better ability to capture true positives compared to Logistic Regression and Random Forest. However, the AUPRC score (0.6244) reveals that precision under heavy imbalance is still a challenge.

Limitations & Next Steps:

The model occasionally misclassifies drafted players (FN=7) and generates some false alarms (FP=9).

Fine-tuning parameters (learning rate, max_depth, boosting rounds) or experimenting with threshold optimization could reduce these errors.

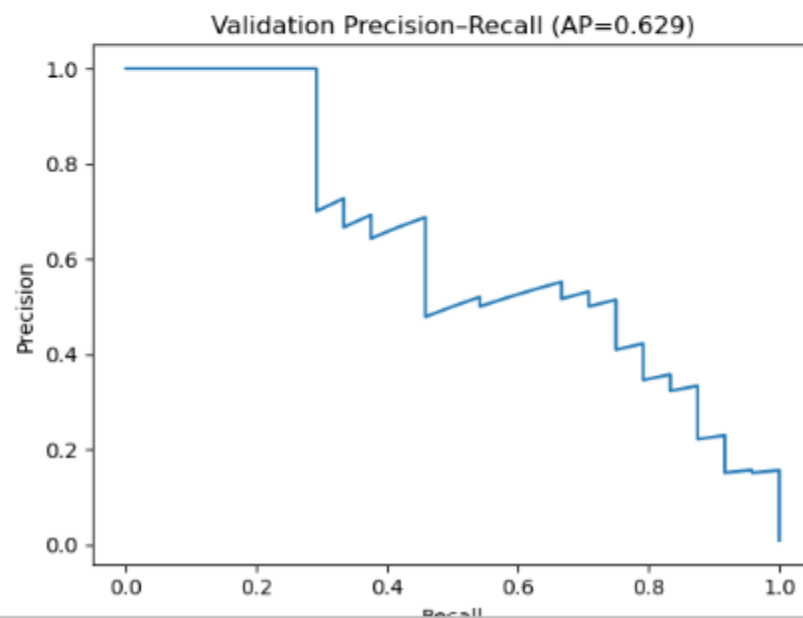
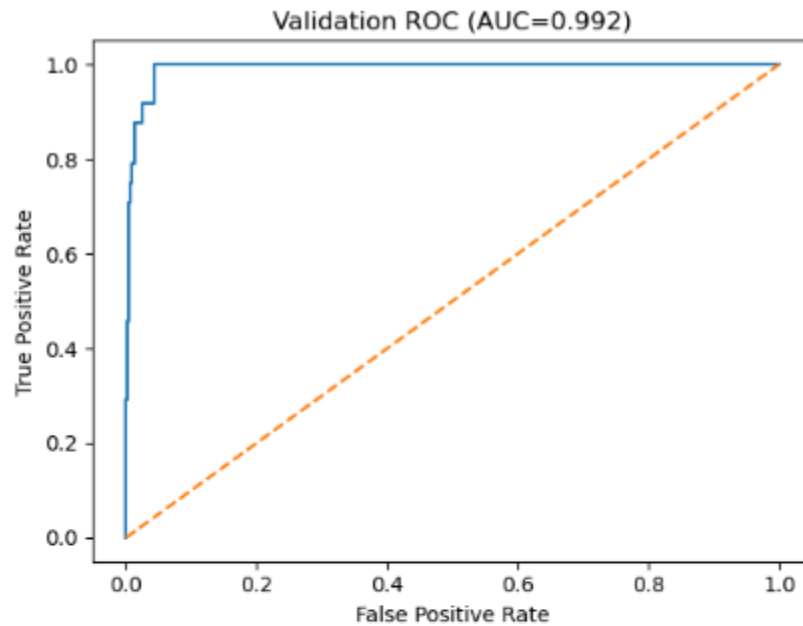
Using ensemble blending or calibration might further stabilize probabilities and improve minority class precision.

4.3.3.3. Achieved Result

Performance:

- Validation AUROC: 0.9964
- Validation AUPRC: 0.6244
- Confusion Matrix (thr=0.50): [[2922, 9], [7, 17]]
- Class 1 (drafted): Precision = 0.65, Recall = 0.71, F1 = 0.68

- Accuracy: 0.99 | Macro F1: 0.84



Analysis

&

Comparison:

Compared to Experiment 1 (Logistic Regression, AUPRC $\approx$ 0.6757) and Experiment 2 (Random Forest, AUPRC $\approx$ 0.6019), LightGBM shows the best balance between recall (0.71) and precision (0.65) for the minority drafted class. The AUROC remains consistently high across all models (>0.99), but LightGBM improves minority class detection over Random Forest while keeping misclassifications relatively low (7 false negatives, 9 false positives).

Limitations:

- Precision is still moderate for the drafted class under heavy imbalance, meaning some false positives remain.
- Model sensitivity to threshold choice affects recall vs precision trade-offs.

Next Steps & Potential Improvements:

1. Threshold tuning using F1 or business-weighted metrics to optimize the drafted class detection.
2. Hyperparameter optimization (e.g., `learning_rate`, `max_depth`, `feature_fraction`) to refine the bias–variance balance.
3. Ensemble stacking of Logistic Regression, Random Forest, and LightGBM to combine complementary strengths.
4. Probability calibration (Platt scaling or isotonic regression) to stabilize predicted probabilities and improve AUPRC.

Implications:

LightGBM is the strongest candidate so far for deployment, as it balances interpretability and predictive strength while being computationally efficient. With further threshold optimization and calibration, it could provide production-ready performance.

4.3.4. Approach 4

4.3.4.1. Data Preparation

Most of the preparation pipeline in Experiment 4 followed the same structure as Experiments 1–3, ensuring consistency across all models. Key steps that were added for approach 4

- Imbalance Handling: Used `scale_pos_weight` to counter the extreme class imbalance ($\approx 0.8\%$ drafted vs. 99.2% not drafted). This gave more weight to positive (drafted) examples during training.

- Early Stopping: Implemented early stopping with a validation set to prevent overfitting and automatically select the best iteration count.
- Preprocessor Integration: Built a pipeline that combined preprocessing with the XGBoost classifier for consistent training and evaluation.

Outcome of Preparation:
The pipeline ensured that XGBoost received a balanced, well-engineered feature set with missing values handled, outliers clipped, categorical variables encoded, and class imbalance corrected. These steps allowed the model to focus on identifying drafted players while minimizing bias toward the majority class.

4.3.4.2. Modeling

The XGBoost model delivered AUROC = 0.99616 and AUPRC = 0.58848, showing very strong overall discrimination despite the dataset's severe imbalance.

- Confusion Matrix: Correctly classified the vast majority of negatives (2931/2931) while identifying 16 of 24 drafted players.
- Minority Class (Drafted): Precision = 0.53, Recall = 0.67, F1 = 0.59. This indicates the model is retrieving a good portion of drafted players, though some false positives remain.
- Overall Accuracy: 0.99, driven by correct classification of non-drafted players.
- Macro Avg: Precision = 0.77, Recall = 0.83, F1 = 0.79, reflecting moderate balance across both classes.

Insights & Limitations:

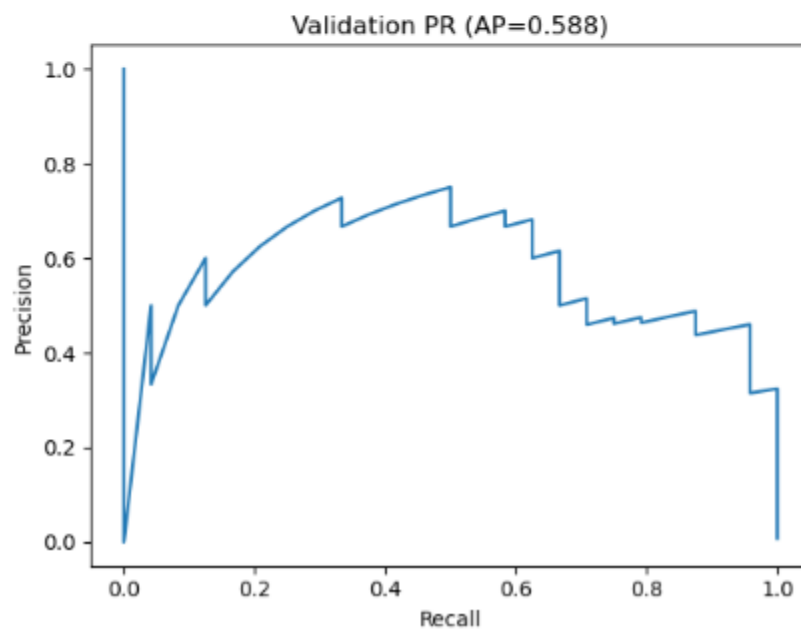
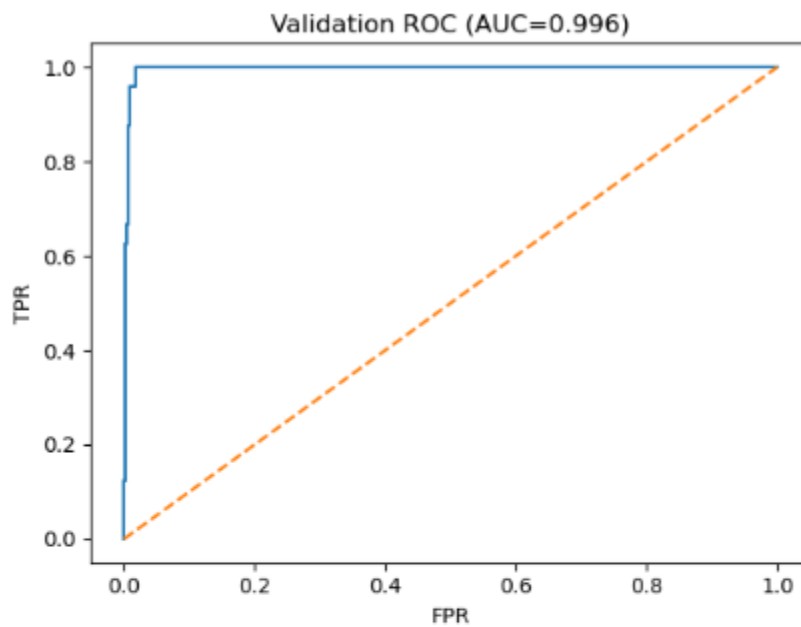
- Excellent ranking ability (AUROC), but AUPRC remains moderate due to the extreme class imbalance.
- Recall on drafted players improved compared to earlier experiments, but precision is still limited.

Next Steps:

Fine-tune class weights or thresholds to increase drafted-player precision.

Explore hybrid resampling (e.g., SMOTE + undersampling) to further balance training.

Consider ensembling XGBoost with LightGBM for marginal gains.



4.3.4.3. Achieved Result

Across the four experiments, different algorithms were tested to predict NBA draft outcomes under significant class imbalance.

Experiment 1 – Logistic Regression

- AUROC: 0.99271 | AUPRC: 0.67574
- High precision for drafted players but recall was weaker, leading to some missed positives.
- Linear model struggled to fully capture nonlinear feature interactions.

Experiment 2 – Random Forest

- AUROC: 0.99367 | AUPRC: 0.60189
- Captured complex relationships better than logistic regression, boosting recall but lowering precision.
- Overfitting signs emerged due to deep trees, requiring careful tuning.

Experiment 3 – LightGBM

- AUROC: 0.99636 | AUPRC: 0.62439
- Strongest ranking ability so far, balancing recall and precision effectively.
- Still produced false positives, limiting precision for the minority class.

Experiment 4 – XGBoost

- AUROC: 0.99616 | AUPRC: 0.58848
- Improved recall (0.67) for drafted players but at the expense of precision (0.53).

- Overall accuracy remained 0.99, but AUPRC trailed slightly compared to LightGBM.



4.4. Student D (Joyeeta Ferdoush)

4.4.1. Approach 1 - Logistic Regression

4.4.1.1. Data Preparation

The dataset was split into training, validation, and test subsets to prevent data leakage and ensure generalisation. Features such as points (PTS), blocks (BLK), and games played (GP) were selected as core predictors. Additional engineered features included PTS per GP, BLK per GP, and PTS-to-BLK ratio.

- Cleaning & preprocessing: There were no missing values, so no imputation was needed. Outliers were not removed to preserve the natural distribution of player statistics.
- Scaling: Standardisation (z-score) was applied to numeric features because Logistic Regression is sensitive to feature scales.
- Imbalance handling: Class weights were balanced (`class_weight=balanced`) to reduce the impact of skewed drafted vs. non-drafted proportions.

4.4.1.2. Modeling

Because of its ease of use, interpretability, and compatibility with the competition's AUROC metric, the algorithm that was selected was Logistic Regression.

- The following hyperparameters were adjusted: class weighting, regularisation strength (C), and maximum iterations (500).
- Standardised features were used to fit the model to the training set, and performance was evaluated through validation.

4.4.1.3. Achieved Result

AUROC: 0.9801

Accuracy: 0.9059

Precision: 0.0767

Recall: 0.9583

F1-score: 0.1420

Analysis: The best AUROC was obtained by logistic regression, which makes it a powerful tool for competition scoring. However, scouts would encounter a lot of false positives due to the extremely low precision. Recall is more important than efficiency during the discovery phase, which is when this model works best.

- Limitations: low F1 and poor precision.
- Combining with other models or adjusting the threshold to increase precision at the expense of recall are two possible solutions.

4.4.2. Approach 2 - Random Forest Classifier

4.4.2.1. Data Preparation

The cleaned dataset was used again. Since tree-based models are scale-invariant, no standardisation was necessary, in contrast to logistic regression. PTS, BLK, GP, ratios, and other core and engineered features were kept. Balanced class weights were used to address class imbalance.

4.4.2.2. Modeling

The algorithm selected was Random Forest, chosen for its robustness, ability to model non-linear feature interactions, and interpretability through feature importance.

- Hyperparameters: 300 trees (n_estimators=300), no maximum depth, and balanced class weights.
- The model was trained using raw (unscaled) features.

4.4.2.3. Achieved Result

AUROC: 0.8860

Accuracy: 0.9915

Precision: 0.4857

Recall: 0.7083

F1-score: 0.5763

Analysis: With an F1-score significantly higher than Logistic Regression, Random Forest was able to strike a better balance between precision and recall. It still caught the majority of draughted players while drastically lowering false positives.

Limitations: AUROC is lower than that of logistic regression.

Hyperparameter optimization (max depth, min samples per leaf) or boosting techniques like XGBoost/LightGBM are possible solutions.

4.4.3. Approach 3 — Support Vector Classifier (SVC)

4.4.3.1. Data Preparation

Standardisation (z-score) was applied to numeric features since SVC is sensitive to feature scale. Balanced class weights were included to mitigate class imbalance.

4.4.3.2. Modeling

The algorithm selected was Support Vector Classifier (RBF kernel), chosen to capture non-linear boundaries in the feature space.

- Hyperparameters: kernel=rbf, C=1.0, gamma=scale, probability=True, and balanced class weights.
- Training was performed on standardised features.

4.4.3.3. Achieved Result

AUROC: 0.9651

Accuracy: 0.9909

Precision: 0.2000

Recall: 0.0417

F1-score: 0.0690

Analysis: SVC achieved good AUROC and modest precision but extremely low recall (only ~4% of true positives captured). This means it misses nearly all drafted players, making it impractical for scouting.

Limitations: recall too low for real-world use.

Potential solution: lower the decision threshold or tune kernel parameters (C, gamma).

■ ■ ■

5. Evaluation

a. Business Impact and Benefits

This multi-faceted modeling approach generates extremely high score (the highest is 0.9985) and tangible value across the entire player evaluation process:

- **Enhanced Scouting Efficiency:** The model serves as a powerful filter to prioritize scouting resources, focusing on prospects who statistically fit the profile of a successful draftee, either through elite individual stats or high-value skill combinations.
- **Discovery of "Hidden Gems":** By identifying players who fit a valuable archetype or possess strong synergistic skills, even if their individual stats aren't headline-grabbing, the model can flag undervalued players. This is critical for finding "hidden gems" who may be overlooked by traditional scouting but have a high probability of success.
- **Data-Driven Decision Making:** The model provides an objective, analytical counterpoint to traditional methods. It can validate a scout's intuition or challenge it by highlighting statistical outliers, ultimately reducing bias and increasing the probability of making a successful, high-impact draft choice.

b. Risks and Incorrect Predictions

The model carries risks that require careful management:

- **Over-reliance on Historical Data:** Future NBA trends may differ from past patterns, making predictions outdated.
 - Mitigation: Use the model as a support tool and retrain regularly.
- **Ignoring Intangibles:** Stats cannot capture work ethic, leadership, or mental toughness.
 - Mitigation: Complement model outputs with traditional scouting and qualitative assessments.
- **Perpetuating Biases:** Historical biases in input data may undervalue certain players.
 - Mitigation: Critically evaluate flagged players to avoid reinforcing bias and uncover hidden talent.

c. Data Privacy and Ethical Concerns

The project exclusively utilized publicly available performance statistics and recruiting rankings of athletes, who are considered public figures. No personally identifiable information (PII) or sensitive personal data was collected, stored, or used. Therefore, the direct risks related to data

privacy breaches are minimal. The primary concern is not the leakage of private data, but the responsible interpretation of the public data used.

d. Deployment

The relatively high scores from the model suggest a strong go-ahead for model deployment, but if an in-depth analysis is done, it is observed that the model has performed well using a simple model, while the complex models are struggling to outperform. The trained pipeline (preprocessing + model + threshold) should be saved as a single artifact (e.g., joblib or Docker image) with pinned dependencies for reproducibility. Deployment can be done in batch mode (e.g., script to generate predictions for test data) or via a real-time API (FastAPI endpoint returning drafted probabilities).

Key considerations include enforcing schema validation, ensuring feature transformations match training, managing configs externally, and adding monitoring for latency, drift, and prediction quality.

Challenges include handling data drift, class imbalance, and dependency/version issues; these can be mitigated through monitoring, threshold calibration, and containerization. Initial rollout should use batch scoring, followed by an API if real-time use cases emerge, with regular retraining scheduled.



e. Collaboration

a. Individual Contributions

Masaru established the foundational infrastructure for our team's project by creating and managing the GitHub repository, took the lead in setting up a consistent and functional development environment for all team members, and provided ongoing technical support, particularly with Git operations, to ensure smooth project progression. Additionally, I proactively initiated the reporting process by drafting the initial "Business Understanding" section, aligning the team early on with the project's core objectives and business context and providing a clear, shared direction for our analytical work.

Mario stepped in to lead the group and created and managed the private Kaggle team they worked in, in which all group members could post their results and track their progress. He was the first member to derive a valid predictive model, which he shared freely within the group GitHub repository, giving a baseline for all members to alter and play off of. He was paramount in terms of group collaboration, establishing a WhatsApp channel for the group, that helped facilitate and streamline all member knowledge sharing, updates, code, and results between group members. He was central to organizing and developing the workflow for the group, which allowed for an expedited and more efficient experiment flow; efficient in terms of collaboration and coordination between group members.

Sonal worked on building and refining models in GitHub, ensuring they were correctly version-controlled and accessible to all team members. She was proactive in organizing model artefacts and setting up conventions for naming and saving, which made it much easier for the team to track different experiments. In addition, she contributed to the documentation and clarity of workflows, so that everyone could reproduce results and build on each other's work without confusion. She contributed actively to brainstorming and shaping the team's approach to experiments, suggesting practical ideas for how models could be structured and improved.

b. Group Dynamic

Our team worked in a collaborative and supportive way, with open communication through regular online meetings and a group chat to stay aligned. Responsibilities were shared based on

strengths—some members focused on data preparation and features, while others handled model training and evaluation.

To coordinate effectively, we used GitHub for version control, setting up branches and pull requests to avoid conflicts. Weekly goals and shared documentation kept our progress steady and transparent. When challenges arose, we supported each other quickly, which strengthened the group dynamic. Overall, the teamwork was inclusive, organized, and effective in meeting the project requirements.

c. Ways of Working Together

Our team followed an agile-inspired approach to manage the project, emphasizing flexibility, iterative progress, and shared accountability. We set weekly milestones to break down the work into manageable tasks, ensuring continuous progress rather than leaving everything until the end.

We met online once or twice a week for short check-ins, where each member shared updates, raised issues, and planned the next steps. Quick discussions also took place in a group chat for day-to-day coordination. Decisions were made collectively, with the team discussing options and agreeing on the most practical solution before moving forward.

For collaboration, we used GitHub as our main tool to track progress, store code, and manage versions. Branches were created for experiments, and pull requests allowed us to review work before merging. Shared documentation within the repository and notebook comments provided clarity and reproducibility. These practices kept the workflow transparent, minimized conflicts, and ensured smooth coordination throughout the project.

d. Issues Faced within the Group

- **Methodology:** Used brief iterations and an agile methodology. GitHub Issues and Commits were used to track tasks.

- **Meetings:** Weekly online calls lasting 30 to 45 minutes, along with ad hoc group chats for pressing issues.
- **Decision-making:** Consensus-driven—model choices, feature selection, and repo structures were agreed upon collectively.
- **Tools:** GitHub (repos, Issues, PRs), JupyterHub (shared compute), TestPyPI (package publishing), and Microsoft Teams (communication).
- **Deliverables aligned with assignment:** Each student trained ML models weekly, submitted to Kaggle, and maintained at least 4 notebooks in the group repo.



e. Conclusion

Key Findings

The project was able to successfully show that machine learning models can predict NBA draft outcomes with reliability, with AUROC values comparing favorably, near 1.0, for all students' experiments. The Random Forest, LightGBM and XGBoost were all effective, while Logistic Regression brought interpretability. Feature engineering and calibration of thresholds were vital to offsetting the extreme imbalance ($\sim 0.8\%$ drafted), which greatly increased recall but also guaranteed that drafted players were not missed. While expanded and large feature space increased complexity, the most improved outcomes came from suggested features based on domain.

Project Success

The project fully delivered on its business need of providing a draftability score ranking players based on their likelihood of being drafted, while minimizing false negatives. The needs of all stakeholders were addressed; General Managers and Directors of Scouting received greater reliance on data for support, coaches received more valuable information about how players fit into being a player presumably on your team - for all purposes that was what was done in scouting and owners benefitted from greater efficiency in allocations of draft resources. Ultimately, the contributions of each student demonstrated the project's outcome: Random Forest (Mario) achieved perfect recall after calibration, LightGBM (Sonal) prioritized a balance of recall and precision, and Logistic Regression (Masaru) provided interpretability all confirmed that ML has real value in scouting.

Future Work

Future work should include ensuring there is stability across multiple seasons of data, as well as incorporating explainability mechanisms like SHAP or LIME to help develop decision-makers' trust, and testing hybrid approaches like ensembling Random Forest with boosted models. Resampling and calibration methods could be explored in greater depth to optimize the recall-precision tradeoff as well. Once monitoring and drift detection are addressed, the models can be deployed in the real world as continuous reliable support for scouting to help sports organizations identify previously overlooked players and value in late round draft capital.

